

Comunicación serial y formato de paquetes

OpenBCI

1. Introducción

El sistema utiliza comunicación serial para recibir los datos provenientes del **OpenBCI Cyton** en una Raspberry Pi.

La comunicación se realiza mediante el puerto:

```
/dev/ttyUSB0
```

utilizando una velocidad de:

```
115200 baudios
```

La conexión se establece mediante la biblioteca `pyserial`:

```
ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
```

Una vez establecida la comunicación, el programa inicializa el OpenBCI y posteriormente inicia la transmisión continua de datos mediante el comando:

```
ser.write(b'b')
```

Durante el streaming, el Cyton transmite paquetes binarios de **33 bytes**.

2. Estructura general de un paquete

Un paquete estándar del OpenBCI Cyton contiene:

```
33 bytes
|
├─ 1 byte   → Header
├─ 1 byte   → Sample Number
└─ 24 bytes → 8 canales EEG
```

└─ 6 bytes → Datos auxiliares
└─ 1 byte → Footer

Visualmente:

Byte 1	Header 1	Datos EEG 24 bytes	AUX 6 bytes	Footer 1
	Sample Number	8 canales × 3 bytes = 24 bytes	6 bytes	

La documentación oficial define los datos EEG como valores **signed de 24 bits**, enviados en formato **MSB first**.

3. Byte por byte

Para entender el paquete es importante recordar que Python utiliza índices comenzando en `0`, mientras que la documentación de OpenBCI describe los bytes comenzando en `1`.

Por lo tanto:

Documentación OpenBCI	Python
Byte 1	packet[0]
Byte 2	packet[1]
Byte 3	packet[2]
...	
Byte 33	packet[32]

Byte 1 — Header

Byte 1
Python: packet[0]
Valor: 0xA0

El primer byte identifica el inicio de un paquete de datos.

El valor esperado es:

```
0xA0
```

En el código se comprueba mediante:

```
if packet1[0] == 0xA0 and packet2[0] == 0xA0:
```

Esta comprobación permite verificar que los datos recibidos comienzan con el encabezado esperado.

4. Byte 2 — Sample Number

```
Byte 2  
Python: packet[1]
```

El segundo byte contiene el número de muestra.

Este valor permite identificar la posición de la muestra dentro de la secuencia de adquisición.

En el código actual este byte no se utiliza directamente porque el programa únicamente comprueba la estructura del paquete y extrae los valores EEG.

La estructura es:

```
Byte 1 → Header  
Byte 2 → Sample Number
```

5. Bytes 3-26 — Datos EEG

Los siguientes 24 bytes contienen los datos de los **8 canales EEG**.

Cada canal utiliza:

```
3 bytes
```

Por lo tanto:

8 canales × 3 bytes = 24 bytes

La distribución es:

Canal	Bytes OpenBCI	Índices Python
Canal 1	3-5	2-4
Canal 2	6-8	5-7
Canal 3	9-11	8-10
Canal 4	12-14	11-13
Canal 5	15-17	14-16
Canal 6	18-20	17-19
Canal 7	21-23	20-22
Canal 8	24-26	23-25

OpenBCI define estos valores como enteros con signo de **24 bits**, con el byte más significativo primero.

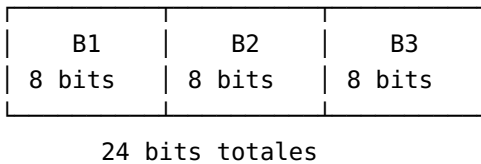
6. Canal EEG de 24 bits

Cada canal ocupa tres bytes.

Por ejemplo, para el canal 1:

Byte 3
Byte 4
Byte 5

Estos tres bytes forman un único número de 24 bits:



El código recibe estos tres bytes mediante:

```
read_24bit_signed(  
    packet[idx],  
    packet[idx+1],  
    packet[idx+2]  
)
```

7. ¿Por qué 24 bits?

El ADC utilizado por el Cyton, el **ADS1299**, proporciona datos de alta resolución para la adquisición de señales bioeléctricas.

Cada muestra EEG se representa mediante:

24 bits

Esto permite representar valores positivos y negativos.

El rango matemático de un entero signed de 24 bits es:

-2^{23} hasta $2^{23} - 1$

es decir:

-8,388,608 hasta 8,388,607

Por esta razón el programa debe interpretar correctamente el bit de signo.

8. Conversión de los tres bytes a un entero

La función utilizada por el proyecto es:

```
def read_24bit_signed(b1, b2, b3):  
    value = (b1 << 16) | (b2 << 8) | b3  
  
    if value & 0x800000:  
        value -= 1 << 24
```

```
return value
```

Primero se combinan los tres bytes.

El primer byte se desplaza 16 posiciones:

```
b1 << 16
```

El segundo byte se desplaza 8 posiciones:

```
b2 << 8
```

El tercer byte permanece en su posición:

```
b3
```

Después se combinan mediante OR bit a bit:

```
value = (b1 << 16) | (b2 << 8) | b3
```

El resultado es un número entero de 24 bits.

9. Interpretación del signo

Los datos del ADS1299 utilizan representación signed.

El bit más significativo de los 24 bits es el bit de signo:

```
bit 23
```

Su máscara es:

```
0x800000
```

Por eso el programa realiza:

```
if value & 0x800000:
```

Si este bit está activo, significa que el valor debe interpretarse como negativo.

Para convertirlo correctamente se utiliza complemento a dos:

```
value -= 1 << 24
```

Como:

```
1 << 24 = 16,777,216
```

se obtiene el valor negativo correspondiente.

Finalmente:

```
return value
```

devuelve el valor entero firmado.

10. Ejemplo conceptual

Supongamos que un canal recibe tres bytes:

```
B1 = 0x01  
B2 = 0x02  
B3 = 0x03
```

El programa realiza:

```
value = (0x01 << 16) | (0x02 << 8) | 0x03
```

Lo que equivale a:

```
0x010203
```

y posteriormente se obtiene el valor decimal correspondiente.

El objetivo de la función no es todavía convertir directamente a microvoltios.

Su función es primero transformar:

```
3 bytes
  ↓
entero signed de 24 bits
```

La conversión a unidades físicas se realiza posteriormente mediante el factor de escala utilizado por el programa.

11. Canales 1-8 en el código

El primer paquete recibido se almacena en:

```
packet1
```

El código recorre los ocho canales:

```
for ch in range(8):
```

`range(8)` genera:

```
0
1
2
3
4
5
6
7
```

Después calcula la posición inicial de cada canal:

```
idx = 2 + ch*3
```

Esto produce:

ch	idx	Canal
0	2	1
1	5	2
2	8	3
3	11	4
4	14	5
5	17	6
6	20	7
7	23	8

Por ejemplo:

```
ch = 0
idx = 2
```

corresponde a:

```
packet1[2]
packet1[3]
packet1[4]
```

que son los tres bytes correspondientes al canal 1.

12. Canales 9–16

El formato estándar de un paquete Cyton contiene ocho canales EEG.

Por esta razón, para obtener los 16 canales utilizando Cyton + Daisy, el programa recibe dos paquetes:

```
packet1 = ser.read(33)
packet2 = ser.read(33)
```

El primer paquete proporciona:

Canales 1–8

y el segundo:

Canales 9–16

La extracción de los canales 9–16 utiliza:

```
for ch in range(8):  
    idx = 2 + ch*3  
  
    data_EEG[ch+8, i] = read_24bit_signed(  
        packet2[idx],  
        packet2[idx+1],  
        packet2[idx+2]  
    )
```

El desplazamiento:

$ch + 8$

convierte los índices:

```
0 → 8  
1 → 9  
2 → 10  
...  
7 → 15
```

que corresponden a:

```
Canal 9  
Canal 10  
Canal 11  
...  
Canal 16
```

13. Bytes 27-32 — AUX

Después de los 24 bytes correspondientes a los ocho canales EEG quedan seis bytes:

Bytes 27-32

Estos bytes son conocidos como:

AUX

La interpretación depende del modo y del byte final del paquete.

En el modo estándar, estos seis bytes pueden contener información del acelerómetro. OpenBCI también contempla diferentes formatos para estos bytes auxiliares.

La estructura general es:

```
Byte 27 → AUX  
Byte 28 → AUX  
Byte 29 → AUX  
Byte 30 → AUX  
Byte 31 → AUX  
Byte 32 → AUX
```

En el código actual estos bytes no se procesan porque el objetivo del proyecto es obtener los datos EEG.

14. Byte 33 — Footer

El último byte del paquete es:

Byte 33
Python: `packet[32]`

Su valor pertenece al rango:

0xC0 — 0xCF

Este byte permite identificar el final del paquete y también indica cómo interpretar los seis bytes auxiliares.

Por ejemplo:

0xC0

corresponde al formato estándar con datos del acelerómetro.

En el código actual no se verifica explícitamente este byte; únicamente se comprueba el encabezado 0xA0 y que el paquete tenga exactamente 33 bytes.

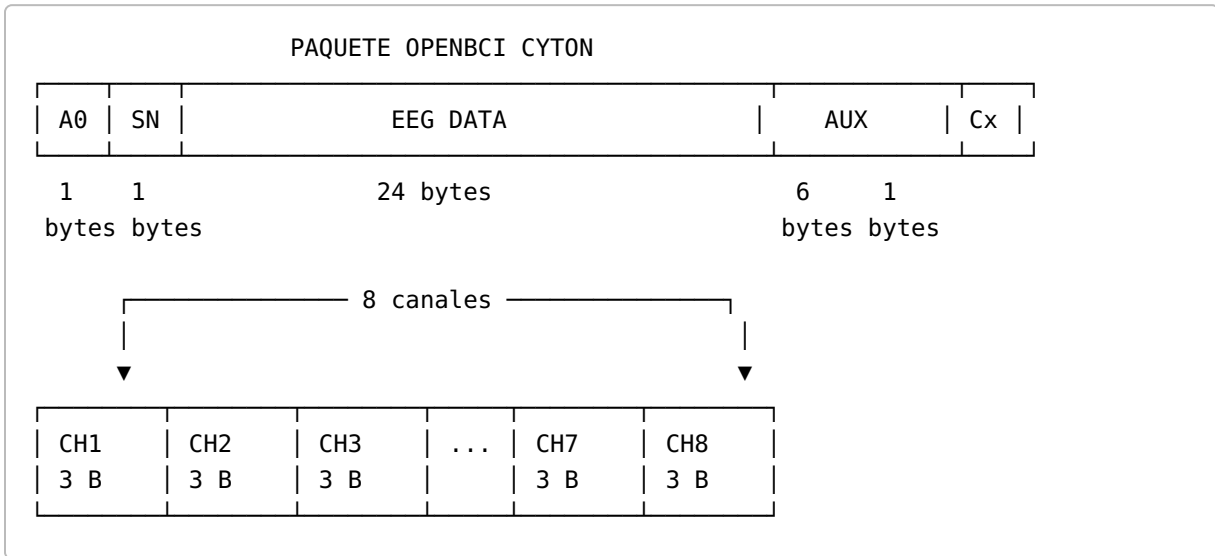
15. Resumen completo de los 33 bytes

La estructura completa puede representarse así:

Byte	Contenido
1	Header = 0xA0
2	Sample Number
3	Canal 1 - byte más significativo
4	Canal 1
5	Canal 1 - byte menos significativo
6	Canal 2 - byte más significativo
7	Canal 2
8	Canal 2 - byte menos significativo
9	Canal 3 - byte más significativo
10	Canal 3
11	Canal 3 - byte menos significativo
12	Canal 4 - byte más significativo
13	Canal 4
14	Canal 4 - byte menos significativo
15	Canal 5 - byte más significativo
16	Canal 5
17	Canal 5 - byte menos significativo
18	Canal 6 - byte más significativo
19	Canal 6
20	Canal 6 - byte menos significativo
21	Canal 7 - byte más significativo
22	Canal 7
23	Canal 7 - byte menos significativo
24	Canal 8 - byte más significativo
25	Canal 8
26	Canal 8 - byte menos significativo
27	AUX
28	AUX

29	AUX
30	AUX
31	AUX
32	AUX
33	Footer = 0xCx

16. Representación visual



17. Cómo utiliza el proyecto estos paquetes

El programa realiza:

```
packet1 = ser.read(33)
packet2 = ser.read(33)
```

Por lo tanto, solicita al puerto serial:

```
33 bytes → packet1
33 bytes → packet2
```

Después comprueba:

```
if len(packet1) == 33 and len(packet2) == 33:
```

para asegurarse de que ambos paquetes están completos.

Posteriormente verifica:

```
if packet1[0] == 0xA0 and packet2[0] == 0xA0:
```

para comprobar que ambos comienzan con el encabezado esperado.

Finalmente:

```
packet1
  ↓
Canales 1–8

packet2
  ↓
Canales 9–16
```

Los datos se almacenan en:

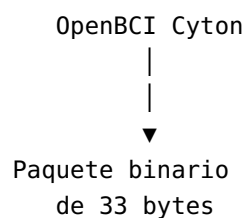
```
data_EEG
```

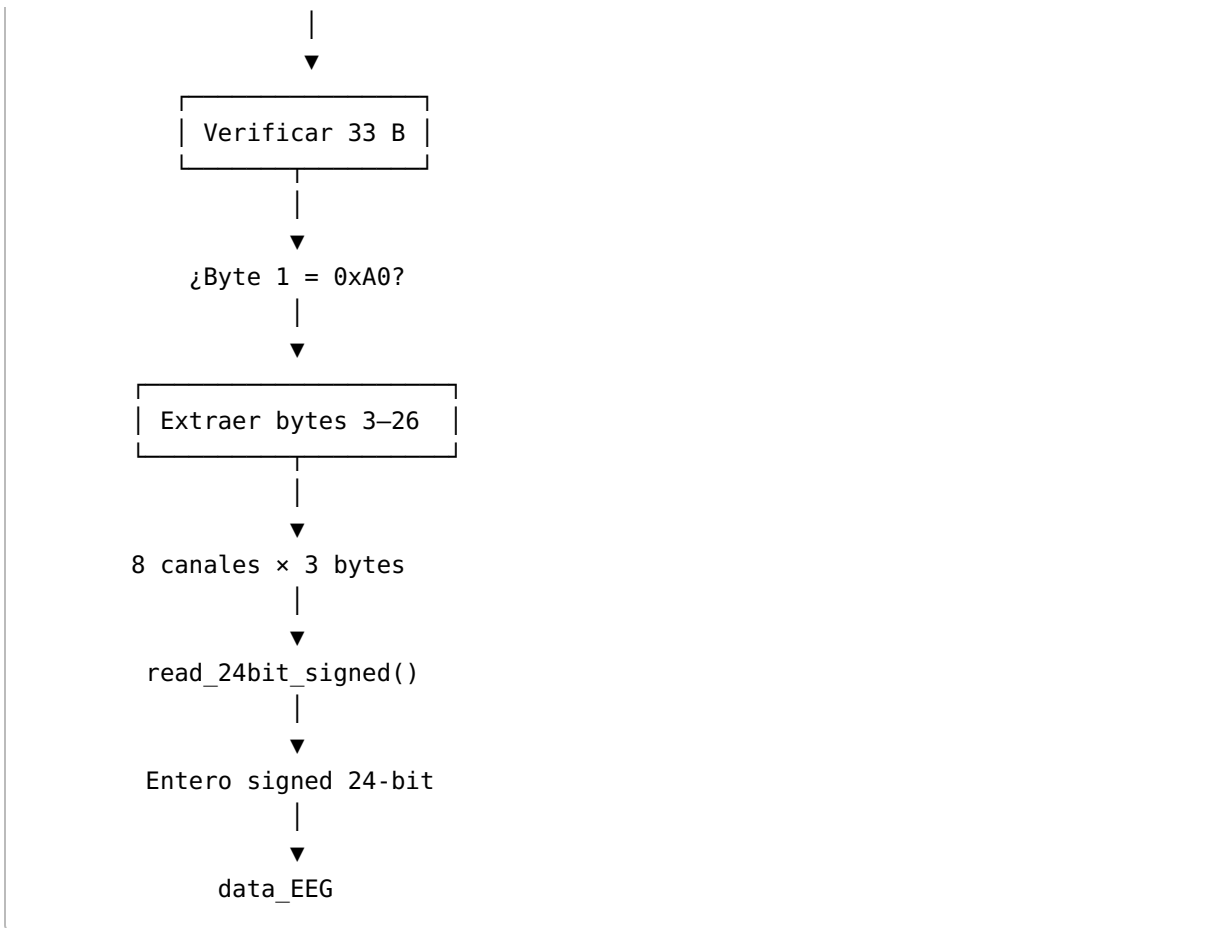
con dimensiones:

```
16 × 1000
```

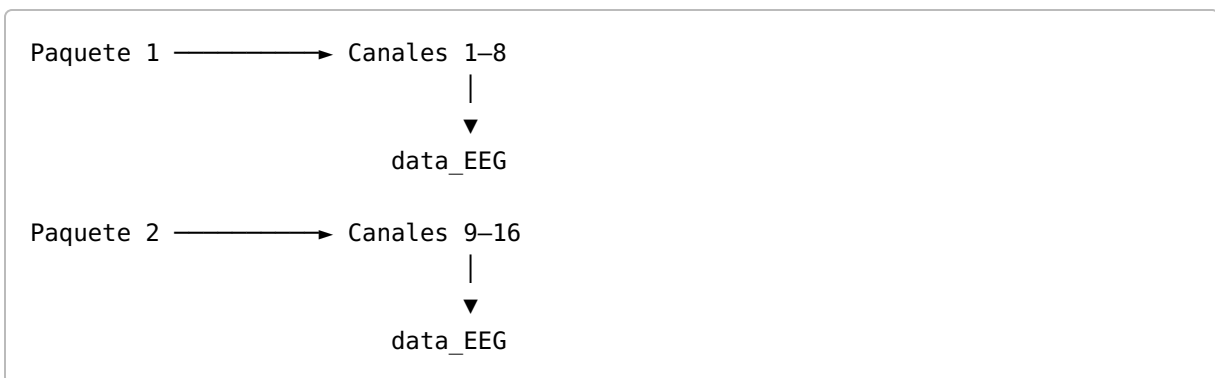
18. Flujo de procesamiento del paquete

El proceso completo puede resumirse como:





Para trabajar con 16 canales:



19. Relación con el código

La parte fundamental del código responsable de interpretar los paquetes es:

```

packet1 = ser.read(33)
packet2 = ser.read(33)

if len(packet1) == 33 and len(packet2) == 33:
    if packet1[0] == 0xA0 and packet2[0] == 0xA0:

        for ch in range(8):
            idx = 2 + ch*3
            data_EEG[ch, i] = read_24bit_signed(
                packet1[idx],
                packet1[idx+1],
                packet1[idx+2]
            )

        for ch in range(8):
            idx = 2 + ch*3
            data_EEG[ch+8, i] = read_24bit_signed(
                packet2[idx],
                packet2[idx+1],
                packet2[idx+2]
            )

```

Esta sección constituye el núcleo de la interpretación de los datos seriales del proyecto.

20. Referencia oficial

La estructura descrita en este documento se basa en la documentación oficial de OpenBCI para el formato de datos del Cyton:

OpenBCI — Cyton Data Format

  OpenBCI Cyton Data Format  <https://docs.openbci.com/Cyton/CytonDataFormat/> 

También puede consultarse la documentación de programación del Cyton:

  OpenBCI Cyton Board Programming Tutorial  <https://docs.openbci.com/Cyton/CytonProgram/> 